# Shizuku Object Firmware Composition 設計メモ

状態: 実装設計メモ（既定設計・実装済・未達の整理）
日付: 2026-10-15
対象リポジトリ: `Shizuku-objfw` (`feat/object-firmware-composition`)
基点 HEAD: `d8c1333` / 関連 commits: `35067dd`, `10534ff`, `d1386ee`

## 1. 目的と基本原則

### 1.1 解決する構造的課題 (現状分析)

1. 全再コンパイルの連鎖: 旧構成では単一 `genrule` (`name = "object_ids_hpp"`) が `internal_headers` に含まれる `generated_object_ids/shizuku/object_ids.hpp` を生成していた。オブジェクト 1 件の追加・変更で全翻訳単位 (TU) の再コンパイルが誘発されていた。
2. 即値埋め込みによるキャッシュ阻害: 旧ツール (`tools/gen_object_ids.py`) が `constexpr uintptr_t <name> = <id>;` として生成していたため、オブジェクトの `.o` 内に ID 即値が直接埋め込まれ、合成 (composition) ごとに再コンパイルが必要であった。
3. 外部合成の不能: 外部リポジトリ (XNO や scratch スケッチ) から `bazel_dep` 経由でオブジェクトを供給し、動的に合成してファームウェア化する仕組みが欠如していた。
4. ビルド設定の外部流出: 利用側が `.bazelrc` に多数のプラットフォーム設定や transition フラグを複製しないとビルドできない状態であった。

### 1.2 確定前提・不変条件 (確定設計)

本設計では以下の不変条件を厳格に保持する（再確認不要の確定事項）:

- **Starlark** が正本: 合成ロジックおよび API 境界は `defs.bzl` を正本とする。
- **API** 基本単位:
  - `shizuku_object(name, srcs, deps)`: firmware/ID 未定の `cc_library` + 型付き marker provider (`ShizukuObjectInfo`)。親子関係や特権を持たず、合成非依存。
  - `shizuku_firmware(name, target, objects, ...)`: firmware 合成時に ID / 親子 / 権限 / chip 設定を決定。マージされた単一 `.a` を出力し、各 SDK の CMake 最終リンクへ渡す。selftest / apps も本 API の利用側へ移行予定。
  - `shizuku_sketch(name, srcs, ...)`: `setup()` / `loop()` を呼ぶ entry と 1 object を作る糖衣 + firmware 生成短縮形。スケッチ公開 label を後から他 firmware の `objects` へ渡せることを設計目標とする。
- **SDK** 境界と **CMake**: pico-sdk を CMake 内へ移すのは設計目標(未実装)。SDK 非依存 Bazel `.a` と常設 CMake dir 再利用を目標とする。C6 先行 → 外部利用確認 → RP2350 移行の順序を遵守。
- ビルド設定の隠蔽 (受入条件): 利用側の `platform/board/C++23/-Og/-g3` 等は transition でマクロ側固定とし、利用側 `.bazelrc` の丸写し・コピーを不要とする。
- シンボルと **canonical C ABI**:
  - 型は既存の `uintptr_t` を維持。
  - canonical exported C ABI は `shizuku_object_id_<name>` (の純粋 C 宣言)。firmware ごとに生成される `.c` (`tools/object_ids.c.in`) が唯一の実体定義。
  - C++ 側 `shizuku::object_id::<name>` は初期化不要の参照エイリアス (`inline constexpr const uintptr_t &`) による段階的互換。静的初期化 (.init_array / ctor) を完全に排除。
  - 旧 global 固定 ID 生成 (`object_ids.hpp`) は移行中であり段階的廃止。
  - レガシー公開シンボル `shizuku_object_id_root = 0` (KERNEL_OBJECT_ID = 0) は互換エイリアスであり、ランタイムのブートスレッド `ROOT_OBJECT = 1` と明確に分離。
- スロット上限と権限所有:
  - 上限 64 スロット。0=KERNEL_OBJECT, 1=ROOT_OBJECT で予約済みのため、userobjects 最大数は 62。
  - 親子関係 (`parents`) および特権フラグ (`privileged_names`) は `shizuku_firmware` が一元所有。
  - ランタイムの `CREATE_OBJECT` 検査規則は維持され、manifest はブート時の初期入力テーブル。
  - XNO は objects 供給元として位置付ける。
- スケッチ実行規約: `setup()` を 1 回呼び出し、`loop()` を反復。反復間に yield を挟む規約。return 終了は不可。
- **RAM** マクロ: `chips/` の `SHIZUKU_RAM_TEXT` 等は SDK ヘッダ非依存とし、リンカスクリプトのセクションディレクティブ (`.iram1.text` 等) のみで制御。SDK 固有ヘッダが必要なコードは SDK component 側へ配置。

## 2. アーキテクチャ設計とインターフェース仕様

### 2.1 シンボル解決とコード生成 (合成非依存 compile action)

オブジェクトは `<stdint.h>` ベースのヘッダ `shizuku/object_id.hpp` または個別宣言マクロ `SHIZUKU_DECLARE_OBJECT_ID` を参照する:

```
// canonical exported C ABI
#include <stdint.h>
#ifdef __cplusplus
extern "C" {
#endif
extern const uintptr_t shizuku_object_id_my_obj;
#ifdef __cplusplus
}
namespace shizuku::object_id {
  inline constexpr const uintptr_t &my_obj = shizuku_object_id_my_obj;
}
#endif
```

`shizuku_firmware` の合成アクションは、指定された `objects` を受け取り以下を行う:

1. 重複検証: オブジェクト名の重複を検査 (`duplicate object error`)。
2. 親子関係検証: 循環依存、自己親、存在しない親の指定を fail-closed で検査。
3. 上限検査: userobjects 総数が最大 62 を超過していないか検査 (`capacity exceeded error`)。
4. 唯一実体コード生成 (**`shizuku_composition_ids.c`**):

```
#include <stdint.h>
const uintptr_t shizuku_object_id_root = 0; // legacy alias (KERNEL_OBJECT_ID)
const uintptr_t shizuku_object_id_my_obj = 2;
// ...
```

各 `shizuku_object` のコンパイルアクションは ID 即値に依存せず未定義シンボル参照 (U) を残すため、異なるファームウェア合成間でも同一の `.o` をキャッシュから再利用可能となる。

### 2.2 Starlark Provider とルール定義 (`defs.bzl`)

#### `ShizukuObjectInfo` Provider

```
ShizukuObjectInfo = provider(
    doc = "Shizuku Object marker and metadata",
    fields = {
        "object_name": "Unique name of the object",
        "entry_point": "Entry function symbol (default: <name>_main)",
        "cc_info": "CcInfo provider containing compiled objects and headers",
    },
)
```

※ 親子関係 (`parent`) および特権フラグ (`privileged`) はオブジェクト単体では決定せず、`shizuku_firmware` が一元的に所有・管理する。

#### `shizuku_object` マクロ

```
def shizuku_object(name, srcs = [], hdrs = [], deps = [], **kwargs):
    # cc_library を生成し、ShizukuObjectInfo を付与したルールでラップ
    ...
```

#### `shizuku_firmware` マクロ / ルール

`target` 引数 (`esp32c6` 先行、`rp2350` 予定) を受け取り、以下を行う:

1. Incoming transition を適用し、ターゲットアーキテクチャのプラットフォーム、C++23、-Og、-g3、strip=never を設定。
2. 依存オブジェクト、カーネル、合成 ID 定義 `.c` を結合して単一のアーカイブ (`libshizuku_firmware.a`) を生成。
3. ESP32-C6 の場合は `modules/esp_idf_support/firmware/build.sh` を通じて `idf.py` リンクを実行し、`.bin` を出力。
4. RP2350 (将来手順) の場合は pico-sdk CMake プロジェクトを通じて最終リンクを実行し、`.uf2` を出力。

**`shizuku_sketch` マクロ**

```
def shizuku_sketch(name, srcs = [], deps = [], target = "esp32c6", **kwargs):
    # 1. setup() / loop() を呼び出す main entry を持つ shizuku_object を生成
    # 2. shizuku_firmware(name = name, target = target, objects = [":" + name + "_obj"], ...) を呼び出す
    ...
```

**2.3 セクション配置 (`SHIZUKU_RAM_TEXT`)**

- ESP32-C6: `#define SHIZUKU_RAM_TEXT __attribute__((section(".iram1.text")))`
- RP2350: `#define SHIZUKU_RAM_TEXT __attribute__((section(".time_critical.text")))`
- ホストテスト / 共通: `#define SHIZUKU_RAM_TEXT`

SDK ヘッダ (`esp_attr.h` や `pico/platform.h`) を include せず、リンカスクリプトのセクション指定のみで RAM 常駐を指定する。

## 3. 現状の到達点 (客観的検証済事実)

客観的なテストログおよびコミット履歴で実証済みの到達点は以下の通りである:

1. **ID 表現基盤とリンク時解決 (commit `35067dd`)**:
   - 純 C ソース (`tools/object_ids.c.in`)、`<stdint.h>`、canonical C ABI (`shizuku_object_id_<name>`) への一本化。
   - コンシューマ側 43 件の既存 global 値コピーを `inline constexpr const uintptr_t &` 参照エイリアスへ完全置換。
   - ホスト Bazel テスト (`//tests/composition_link:compile_once_two_links_test`):
     - `sample_object.o` のハッシュ不変 (`709fb0d432df84ef57ecd9d8853f7a4b712dd4706fe19eaf6e5a8e6ab2675618`)。
     - 動的初期化関数不在 assert (Mach-O: `otool -l`, ELF: `readelf -S`, `nm`)。
     - 純 C コンシューマ (`test_c_consumer.c`, C17) による全 ID 一致 assert。
     - 2 つの異なる ID 割当リンク (Alpha: sample=2/target=3, Beta: sample=4/target=5) の成功。
2. **C6 API スケルトンとマニフェスト管理 (commits `10534ff`, `d1386ee`)**:
   - `defs.bzl` における `ShizukuObjectInfo` provider、transition、manifest 親子 cyclecheck、自己親禁止、上限 62 検査の実装。
   - SDK `build.sh` への第 5 引数 (固定 builddir 指定) の追加。

## 4. 未達・未検証事項と今後の受入条件 (明示的境界)

以下の事項は現時点で「未達」または「未検証」であり、完了扱いにしてはならない:

1. ランタイム起動 **Glue** および **Yield** 接続 (未達):
   - C6 manifest から `CREATE_OBJECT` および sketch entry を起動する本番 glue コードは未実装。
   - C6 sketch loop 間の yield は未接続。
   - 事実関係: リポジトリ内に `shizuku::services::yield()` という名前空間は存在しない (旧記述の誤り)。実在する `shizuku::kernel_services::yield()` (`source/kernel/services.cpp:34`) は privileged internal (`ARCH::current_priv()`) かつ ARM (`#if defined(__arm__)`, L6) 実装限定であり、C6 (RISC-V) 向けの public yield API は存在しない。
2. 常設 **builddir** の **incremental reuse** 実証 (未検証):
   - SDK `build.sh` 側で毎回 `idf.py set-target` が実行されており `CMakeCache.txt` がクリアされるため、真の incremental 再利用は実証されていない。
   - global 旧 ID 定義の混入防止および ODR 境界の完全分離は未完成。
3. 外部 **`.bazelrc`** なしビルド (検証中 / 受入条件):
   - 外部 scratch リポジトリから `.bazelrc` コピー不要で `shizuku_sketch` / `shizuku_firmware` がビルドできることは受入条件である。
   - 現状: 検証ジョブ `f2e769f2-4068-4df3-8536-232bba681b84` が実行中。直前の検証 `48b04e75` は `defs.bzl:468` の辞書フォーマットエラーで exit 1 となっており、未達。
4. 新 **API** による実機バイナリ生成と **RAM** 配置 (未達):
   - 新 API 経由での C6 `.bin` 成功、`.map` による RAM 配置確認、RP2350 `.uf2` 成功はすべて未達。
   - 既存 `.rbuild-out` に存在する C6/C5 バイナリは現 HEAD のビルド入力・ログとの対応が未検証の旧 snapshot であり、新 API の成功証拠ではない。
   - `chips/` の RAM マクロ 3 ファイル (`chips/esp32_c6/chip.hpp`, `chips/esp32_c6/trap.cpp`, `chips/rp2350/chip.hpp`) の差分は未コミットであり、`.map` 検証も未実施。

5. **RP2350 移行 (未着手)**:
   - pico-sdk の CMake 移行および RP2350 向け `shizuku_firmware` リンクは未着手。C6 の受入完了後に着手予定。

## 5. 検証手順案 (提案 / Future Work)

以下のコマンドは将来の検証手順案であり、現時点での実行・成功を主張するものではない。

### 案 1: 外部 scratch における no-.bazelrc ビルド検証

```
# 外部ディレクトリ作成と MODULE.bazel / BUILD.bazel 配置後 (案)
# .bazelrc は作成しない (0 バイト / 不在)
bazel build //:my_sketch_archive
# 期待値: transition により RV32 target / C++23 で .a が生成されること
```

### 案 2: RAM セクション配置の .map 検証

```
# ESP-IDF 最終リンク後の .map 検査 (案)
grep -E "shizuku_.*\.iram1" build/shizuku_esp32c6.map
# 期待値: SHIZUKU_RAM_TEXT を付与したシンボルが .iram1.text に配置されていること
```